C#: syntactic sugar….almost

Giovanni Lagorio & Maura Cerioli
Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi
University of Genova
TAP 2018-19

var

- Solo per dichiarare variabili locali, il tipo (inferito) è quello dell'inizializzatore
- Esempio:

```
int i = 0;
var i = 0; // equivalenti, finché non cambia inizializz.
```

- E' chiaramente molto comoda con i generici...

```
List<List<string>> lls = new List<List<string>>();
var lls = new List<List<string>>();
```

- Non solo per i pigri, essenziale in alcuni casi

Proprietà (Properties)

- Metodi get/set che invocati attraverso la sintassi di accesso a campo, per esempio, *exp*.P = 3;
  - Usate per caching, lazy loading, ecc.
  - E' possibile specificare solo il getter o il setter
  - Per il setter, **value** indica il valore "scritto"
- Al client "sembrano" accessi a campo: è ragionevole usarle solo se getter/setter efficienti
- In quanto metodi, possono essere virtual, abstract, comparire nelle interfacce
- Uno dei due può avere un modificatore di accesso più restrittivo (per es. il setter private)

Uso ovvio

```
public class C {
    private int _i;

    public int I {
        get {
            return this._i;
        }
        set {
            this._i = value;
        }
    }
}
```

```
C aC = new C();
aC.I = 41;
++aC.I; // !!!
Console.WriteLine(aC.I);
    // stampa 42
```

Auto-Properties

- Come una property, ma il campo nascosto è generato automagicamente dal compilatore
  - non se ne conosce il nome

```
public class C{
 public int I { get; set; }
 public int K { get; }
 public int Z { get; private set; }
}
```

- Con ReSharper la trasformazione fra una versione e l'altra richiede solo un ALT Return …
- Le Auto-Properties si possono inizializzare
  - sintassi analoga ai campi

```
public int I { get; set; } = 37;
```

  - come per i campi
    - inizializzazioni eseguite nell'ordine in cui sono scritte
    - nell'espressione di inizializzazione non si può far riferimento al this

Properties senza campo - Esempio

```
public class Person {
    public int YearOfBirth { get; set; }
    public int Age {
        get {
            return DateTime.Now.Year - this.YearOfBirth;
        }
    }
}
// ....
var p = new Person { YearOfBirth = 1973 };
Console.WriteLine(p.Age);
```

Scorciatoie varie...

```
var array = new[] { "ciao", "mondo" };
var anA = new A(3) { G=3 };
var anotherA = new A { F=5, G=3 };
var ls = new List<string>()
                { "qui", "quo", "qua" };
var d = new Dictionary<int, string>()
            { { 1, "pippo" }, { 2, "pluto" } };
var d1 = new Dictionary<int, string>()
            { [1] = "pippo" }, { [2]="pluto" } };
```

Operator overloading

- In molti linguaggi alcuni operatori sono overloaded (per es. quelli aritmetici)
- C# permette di definire il comportamento di molti operatori, quando (almeno) un operando è di un tipo definito dall'utente
  - Non si possono aggiungere operatori o variarne la precedenza
  - Non si può ridefinire l'assegnazione (nemmeno struct)
- Gli operatori vanno definiti **public static** e hanno come nome **operator *op***
  - Per chi conosce C++: "this" non è l'operando sinistro, negli operatori binari servono due parametri

Esempio di uso dell'overloading

```
DateTime x = DateTime.Now;
DateTime y = new DateTime(1982, 1, 1);
TimeSpan t = x - y;
DateTime k = x + t;
```

DateTime TimeSpan sono due tipi (struct) di sistema.

Operator overloading - esempio

```
public class C {
    public static C operator +(C x, int y) {
        return null;
    }
}

public class Program {
    public static void Main(string[] args) {
        var c = new C();
        C foo = c + 3;
        c += 27; // (!!!) come scrivere c = c+27;
    }
}
```

Operator overloading

- Alcuni operatori (== e !=, > e <, >= e <=) vanno a coppie, se si definisce uno, si deve definire anche l'altro
- Se si definisce == è buona norma ridefinire anche Equals e GetHashCode

Parentesi su Equals

```
class C {
    private int x, y;
    public override bool Equals(object obj) {
        if (obj == null)
            return false;
                    // secondo me sarebbe più sensato:
                    // throw new NullReferenceException();
                    // (ma il contratto di Object.Equals dice esplicitamente
                    // di restituire false in questi casi e che Equals non deve
                    // mai sollevare eccezioni)
        if (obj.GetType()==typeof(C)) {
            C other = (C) obj;
            return this.x == other.x && this.y == other.y;
        }
        return false;
    }
    public override int GetHashCode() {
        return …
    }
}
```

User-defined conversions

Il tipo sorgente o destinazione deve corrispondere al tipo che contiene la conversione

```
public class C {
    public static implicit operator C(int i) {
        return null;
    }
    public static implicit operator int(C from) {
        return 1;
    }
    public static explicit operator string(C from) {
        return "a";
    }
}
```

```
        ...
        C c = 42;
        int i = c;
        string s = (string)c;
```

Indexer

- Misto fra proprietà e overloading di [] (una classe può dichiarare diversi indexer, in overload)
- var h = new Dictionary<string, int>();
  h["ciao"] = 7;
- public class C {
  private int _x;
  public int this[double a, double b] {
  get {return 42;}
  set {this._x = value;}
  }
  }

  ...

  var c = new C();
  c[4, 2.0] = 7;
  int i = c[Math.PI, 0];

```
public class BankAccount {
  Dictionary<DateTime, double> _m = new ...
  public double this[DateTime d] {
    get { return this._m[d]; }
    set { this._m[d] = value; }
  }
}
...
var b = new BankAccount();
var d = new DateTime(2013,10,24);
...
Console.WriteLine("Movimento in data {0}={1}",
                                  d, b[d]);
```

- Si possono usare per inizializzare

```
var conversion = new Dictionary<string,int>{
   ["diciotto"] = 18,
   ["diciannove"] = 19,
  /*...*/};
```

dynamic

- In C# 4 è arrivato il tipo **dynamic** che corrisponde al tipo «staticamente dinamico» (=un tipo che richiede **binding dinamico**)
  - Molto elegante, ma dietro le quinte diventa Object e viene usata la reflection a palate (= lento, come usare un linguaggio dinamicamente tipato)
    - È un uso furbo/ottimizzato della reflection, si vedano gli expression tree (di cui parleremo in seguito…) e il DLR
  - Comodo per certi scopi, per es., interfacciarsi a linguaggi dinamicamente tipati
- dynamic è TOTALMENTE diverso da var

Esempio con IronPython

```
var engine = Python.CreateEngine();
var scope = engine.CreateScope();
scope.SetVariable("foo", 21);
var source = engine.CreateScriptSourceFromString(@"
class X(object):
    pass
answer = X()
answer.Value = foo*2");
source.Execute(scope);
var answer = scope.GetVariable("answer");
Console.WriteLine("Answer = {0}.", answer.Value);
```